



■ CS177 Bioinformatics: Software Development and Applications

Lecture 7: Introduction to Deep Learning

Part 2. RNN and Autoencoder

Jie Zheng (郑杰)

PhD, Associate Professor

School of Information Science and Technology (SIST), ShanghaiTech University

Spring, 2025



Learning objectives



- After taking this lecture, you should be able to:
 - Describe the fundamental concepts and ideas behind RNNs
 - Describe some well-known applications of RNNs
 - Explain how to train RNN models and handle the problem of unstable gradients
 - Explain how to handle the problem of limited short-term memory, by LSTM and GRU cells
 - Explain the basic idea behind Autoencoders from perspective of efficient data representations
 - Explain the relation between Autoencoder and PCA
 - Describe the main ideas of a few variants of Autoencoders





Recurrent Neural Networks (RNNs)





Motivation: Different types of sequence learning/recognition/prediction problems

- Sequence classification
 - A sequence to a label, e.g. recognizing a single spoken word
- Segment classification
 - Segments in a sequence correspond to labels
- Temporal classification
 - General case: sequence (input) to sequence (label) modeling
 - No clue about where input or label starts
- Many applications
 - Natural language processing (NLP) systems
 - Bioinformatics analysis of DNA/RNA/protein sequences
 - Analysis and prediction of time-series data, e.g. stock prices

Foreign Minister.



FOREIGN MINISTER.



THE SOUND OF

Source: Alex Graves, 2012



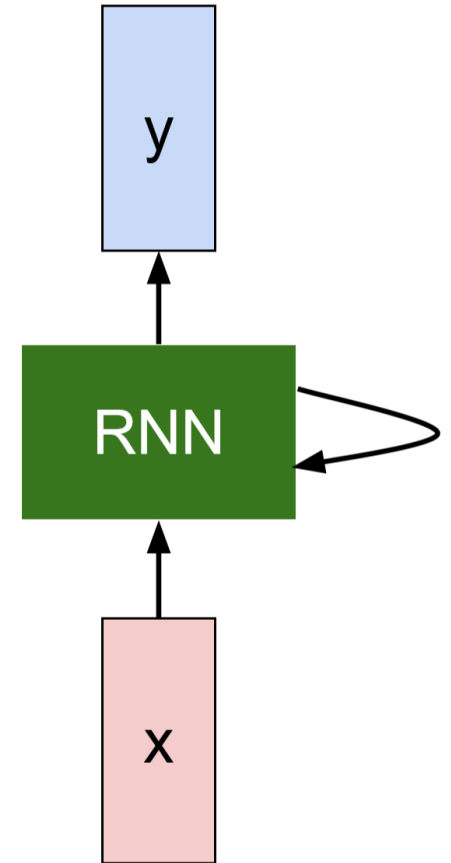
Recurrent Neural Networks (RNNs)

- We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrent formula** at every time step:

$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state some function with parameters W old state input vector at some time step

- Key idea: RNNs have an “internal state” that is updated as a sequence is processed
- Note: The same function and the same set of parameters are used at every time step



■ (Simple) Vanilla RNN



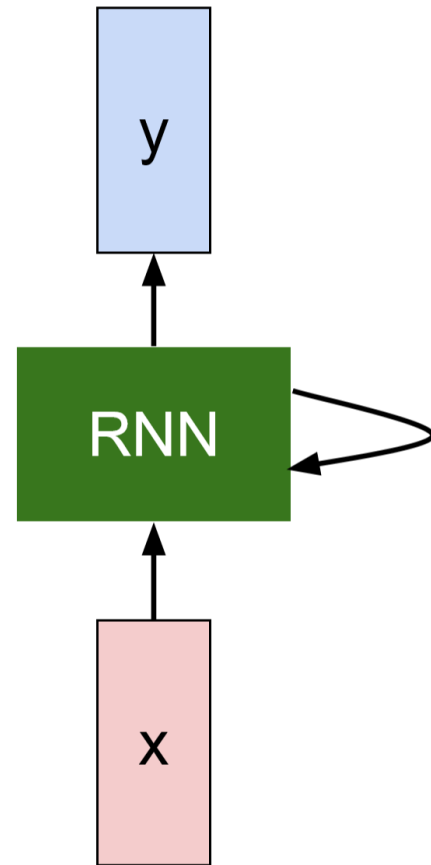
- The state consists of a single “hidden” vector $\mathbf{h}$:

$$h_t = f_W(h_{t-1}, x_t)$$



$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$

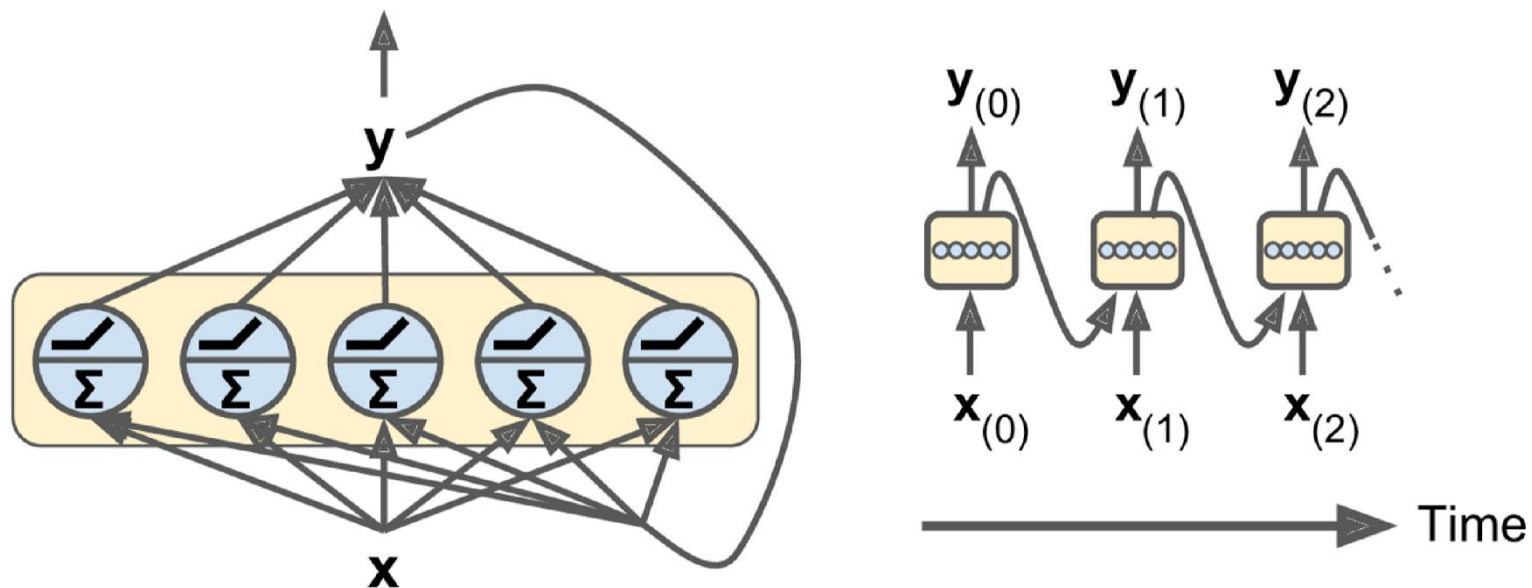
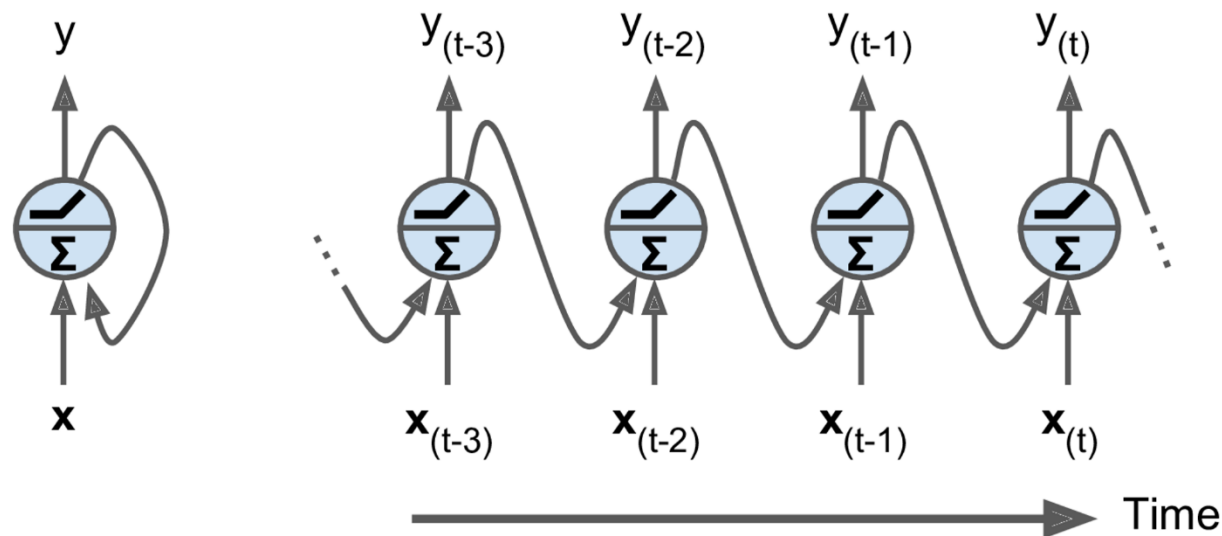


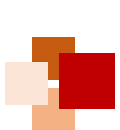
Sometimes it is called an “Elman RNN”
after Prof. Jeffrey Elman





Unrolling the network through time

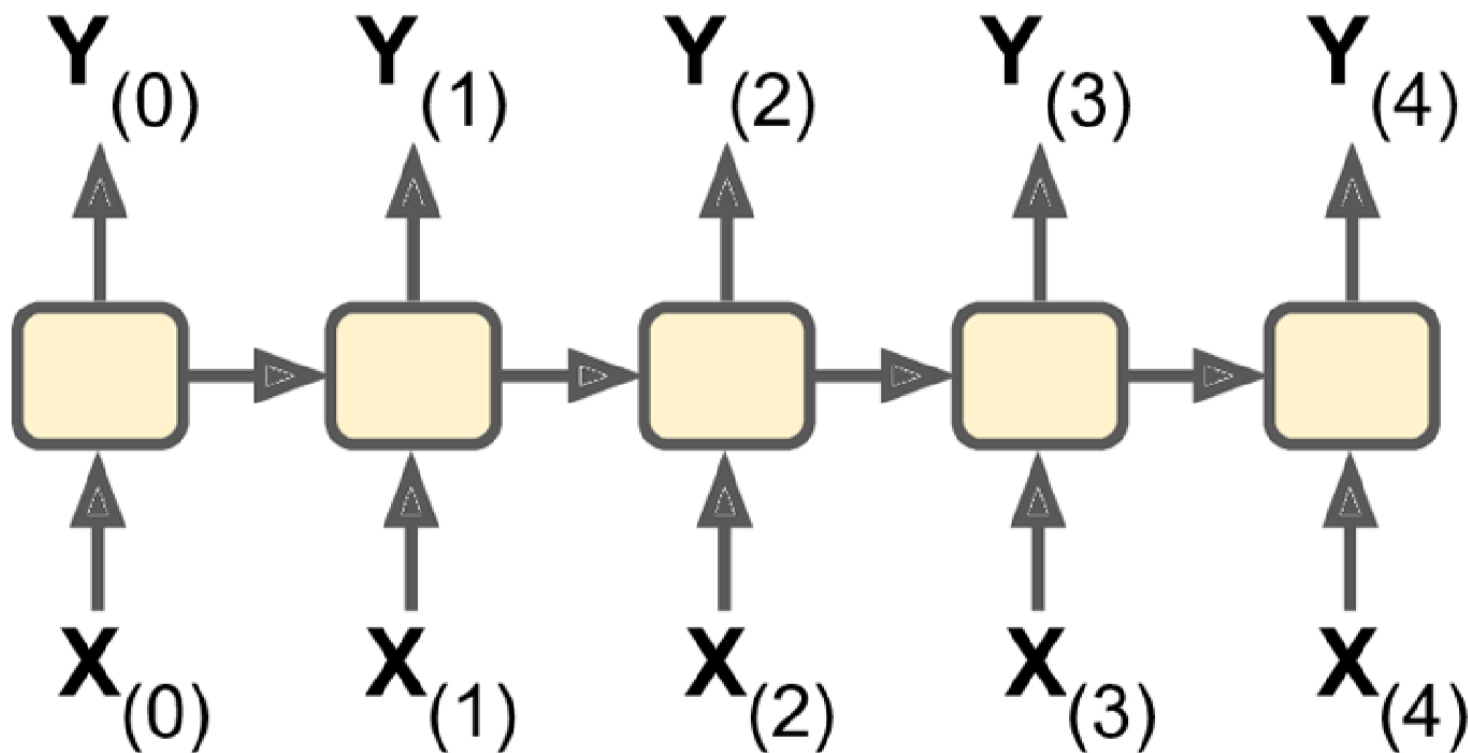




Sequence to sequence (many to many)

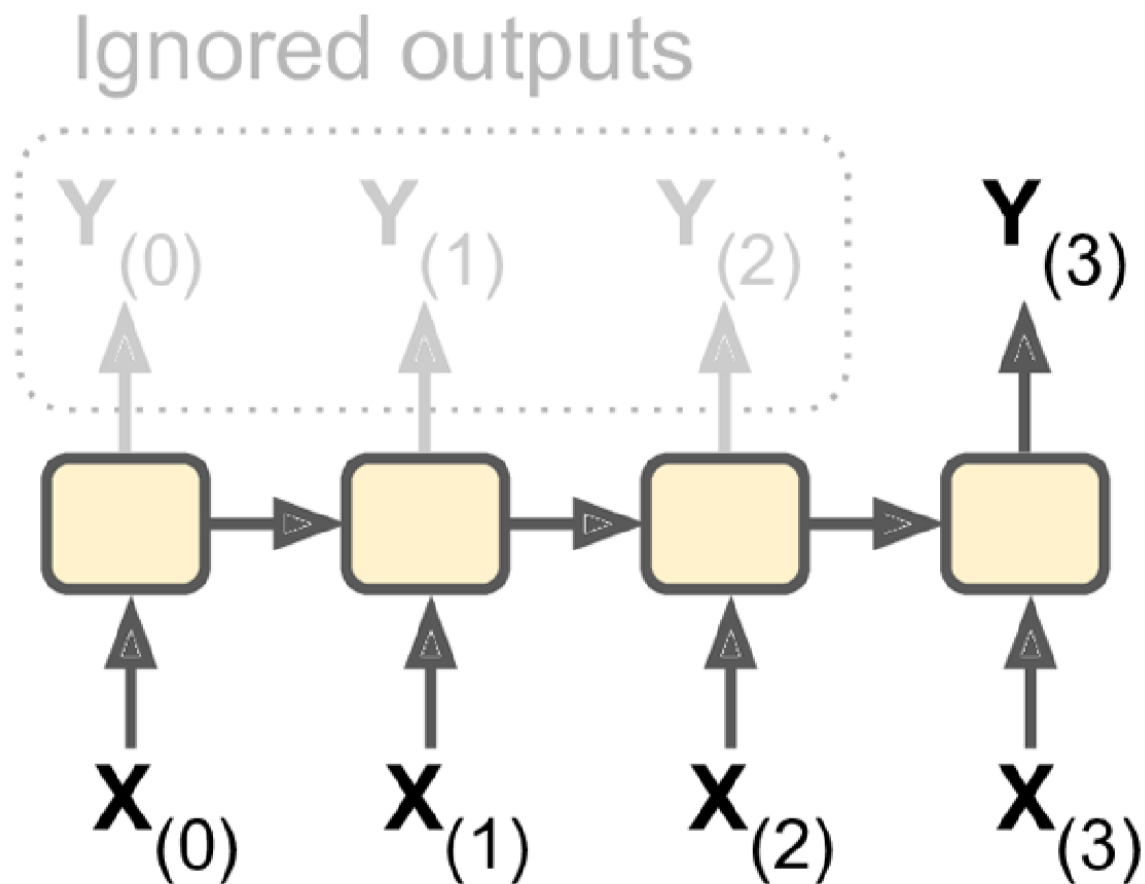


- **Example (Predicting stock prices):** Feed it the prices over the last N days, output the prices shifted by m days into the future



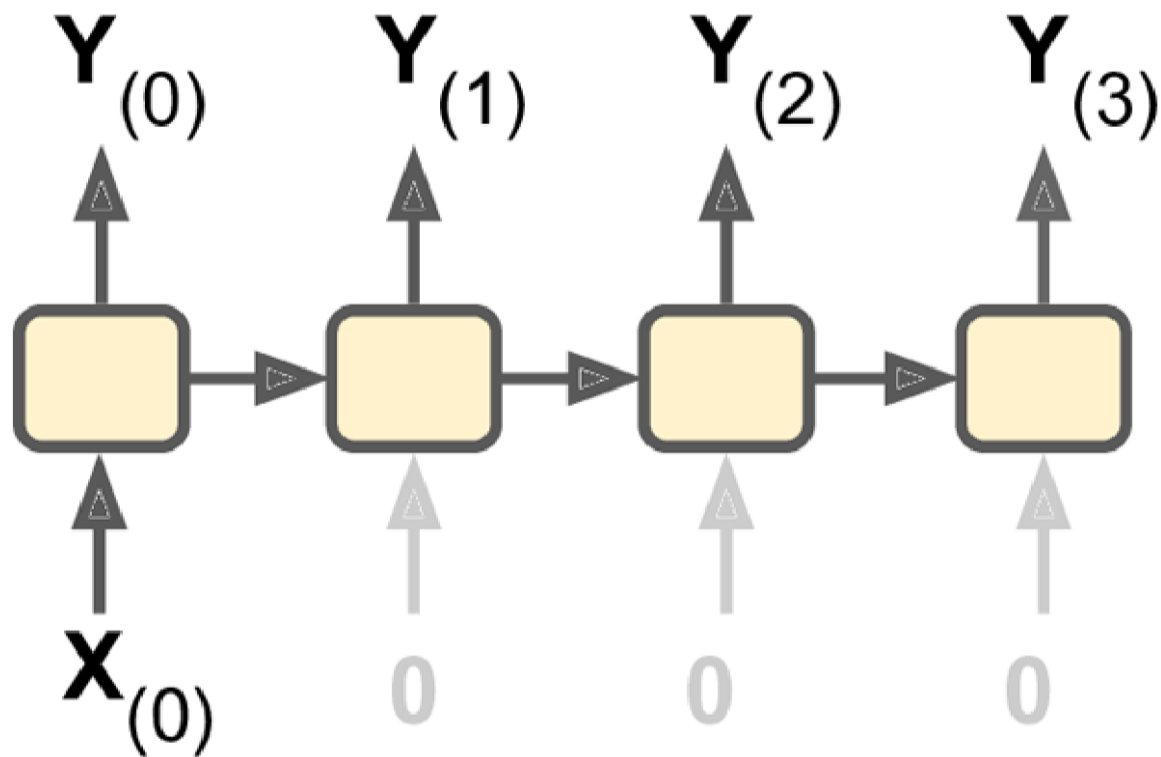
Sequence-to-vector (many to one)

- **Example (sentiment analysis):** Feed the network a sequence of words corresponding to a movie review, and it outputs a sentiment score (e.g. from -1 [hate] to +1 [love])



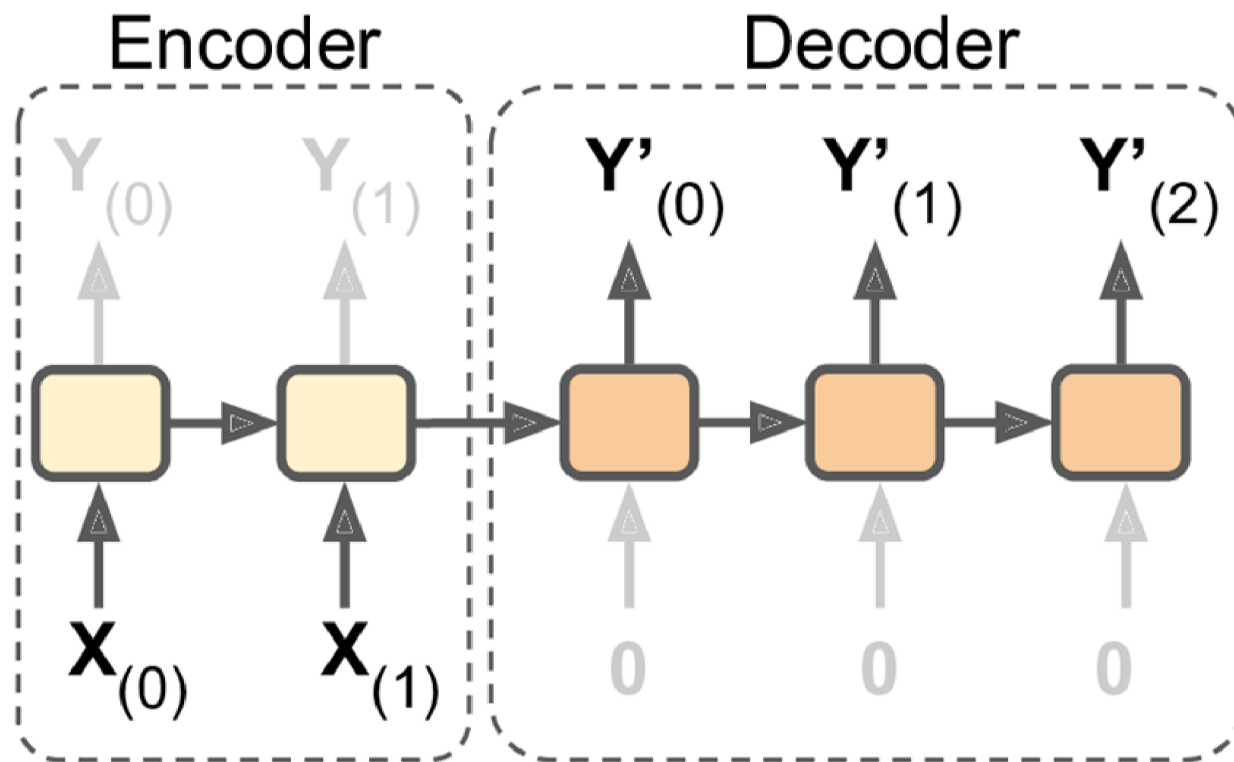
Vector-to-sequence (one to many)

- **Example (image caption):** Input an image, and output a sequence of words as a caption for that image



Delayed sequence-to-sequence (Encoder-Decoder)

- **Example (machine translation):** Input a sentence in English, and the network translates it into a sentence in German
- The two-step model of encoder-decoder works better than translation on the fly, because the last words of a sentence could affect the first words of the translation



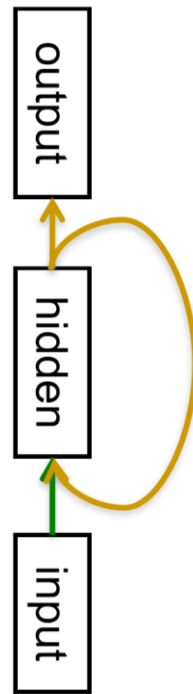
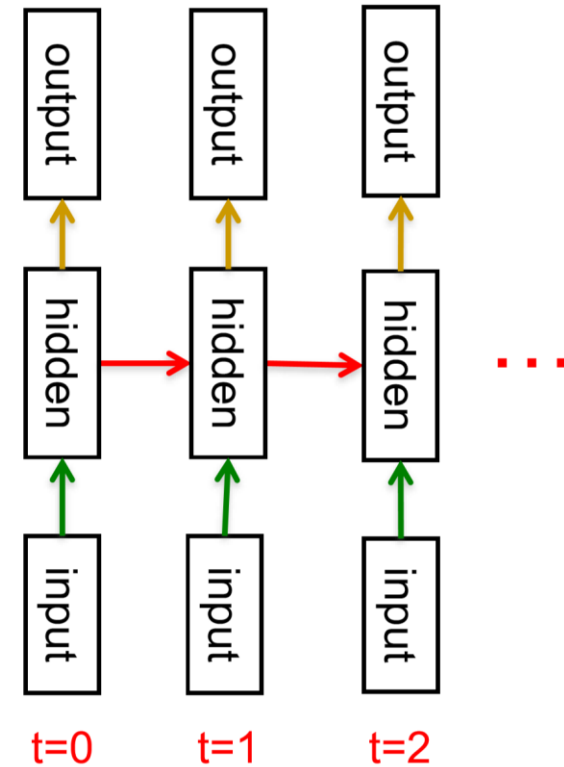


Training RNNs



Challenge of training RNNs

- Backpropagation is designed for feedforward networks
- What would it mean to backpropagate through a recurrent network?
 - Error signal would have to travel back in time

Feed-forward
networksRecurrent
networks

time →

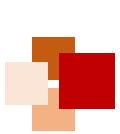


Backpropagation through time (BPTT)



- We can think of the recurrent network as a layered feedforward network with shared weights, and then train the feedforward network with the weight constraints
- We can also think of this training algorithm in the time domain:
 - The forward pass builds up a stack of the activities of all the units at each time step
 - The backward pass peels activities off the stack to compute the error derivatives (i.e. gradients) at each time step
 - After the backward pass, we add together the derivatives at all the different time steps to update the model parameters

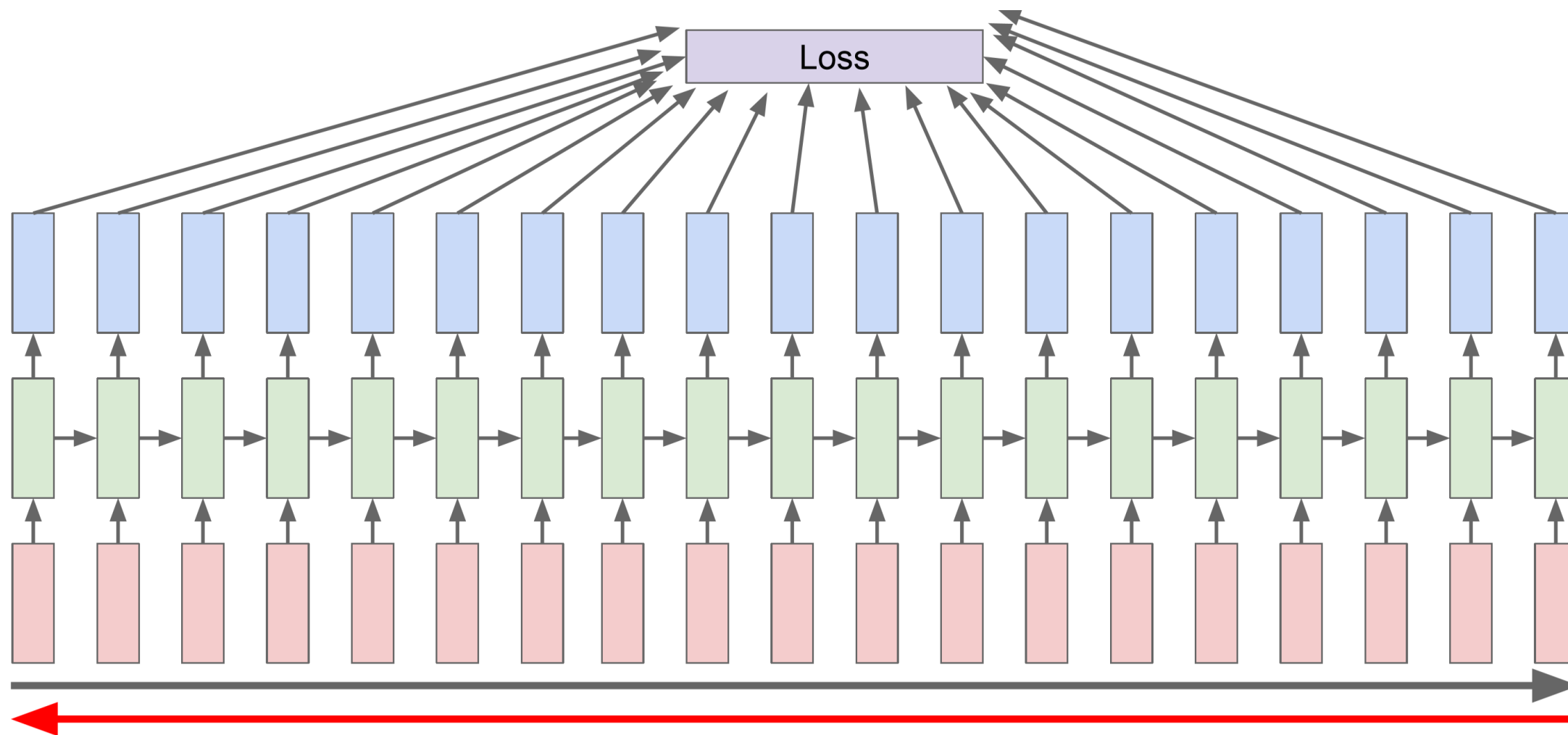




Backpropagation through time (BPTT)



- Forward through entire sequence to compute loss, then backward through the entire sequence to compute the gradients

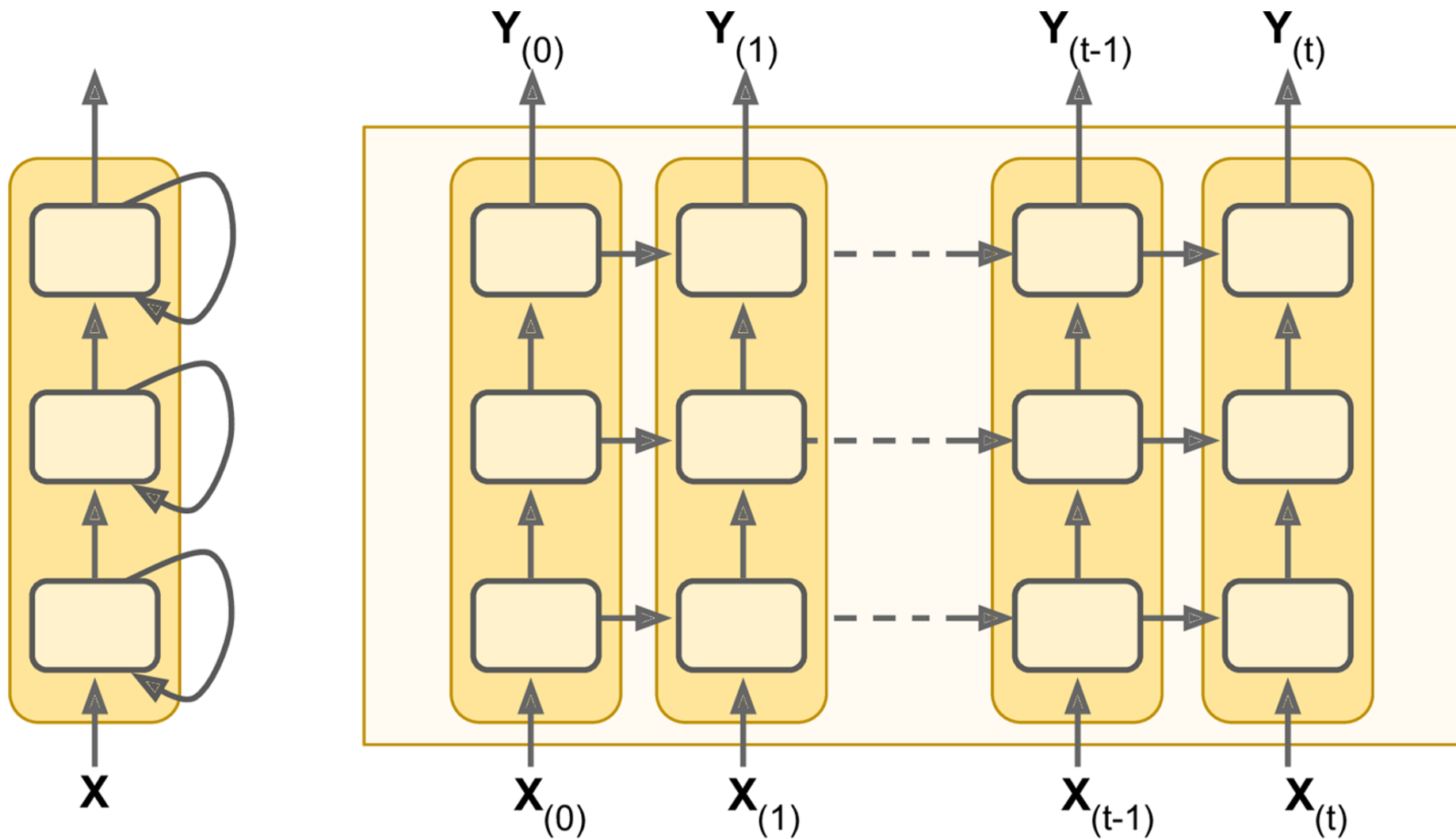




Deep RNN



- Stacking multiple layers of recurrent cells gives a **deep RNN**





Vanishing/exploding gradients problems

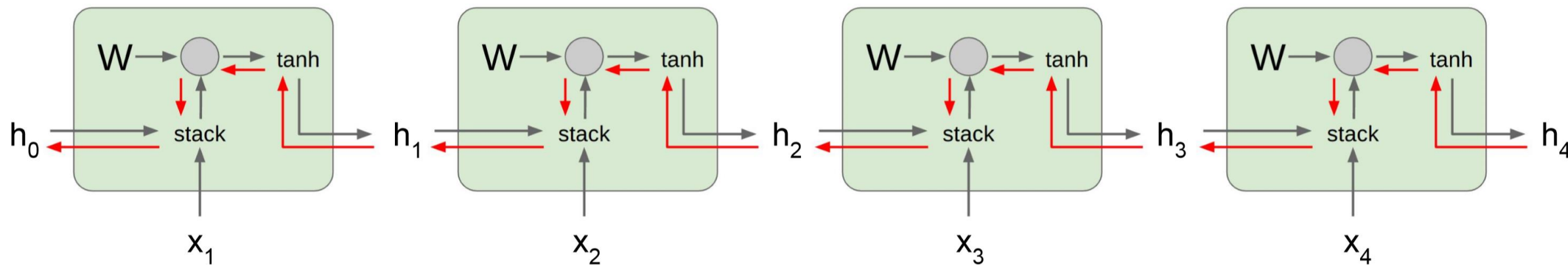


- What happens to the magnitude of the gradients as we backpropagate through many layers?
 - If the weights are small, the gradients shrink exponentially
 - If the weights are big, the gradients grow exponentially
- Typical feedforward neural networks can cope with these exponential effects because they only have a few hidden layers
- In an RNN trained on **long sequences** (e.g. 100 time steps) the gradients can easily explode or vanish
 - We can avoid this by initializing the weights very carefully
- Even with good initial weights, it is very hard to detect that the current target output depends on an input from many time steps ago
 - RNNs have difficulty dealing with **long-range dependencies**





Vanishing/exploding gradients problems



Computing gradient of h_0 involves many factors of W (and repeated $\tanh$)

Largest singular value > 1 :
Exploding gradients



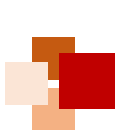
Gradient clipping: Scale gradient if its norm is too big

Largest singular value < 1 :
Vanishing gradients



Change RNN architecture





Gradient clipping



- To lessen the **exploding gradients problem**, we can simply clip the gradients during backpropagation, so that they cannot exceed some threshold. This is called **Gradient Clipping**
- Pseudocode:

```
grad_norm = np.sum(grad * grad)
if grad_norm > threshold:
    grad *= (threshold / grad_norm)
```

- Python code using TensorFlow API:

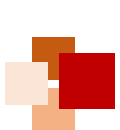
```
threshold = 1.0
optimizer = tf.train.GradientDescentOptimizer(learning_rate)
grads_and_vars = optimizer.compute_gradients(loss)
capped_gvs = [(tf.clip_by_value(grad, -threshold, threshold), var)
               for grad, var in grads_and_vars]
training_op = optimizer.apply_gradients(capped_gvs)
```





Long Short Term Memory (LSTM)

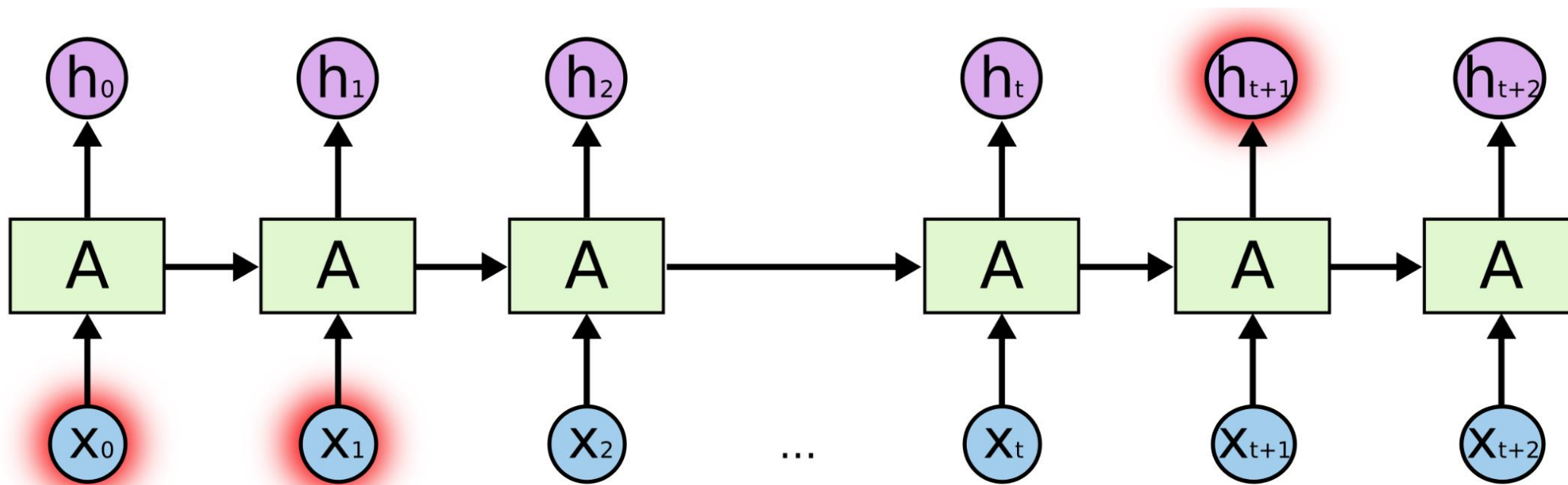




Key challenge



- Learning long-term dependencies is hard



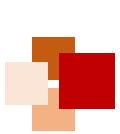


Long Short Term Memory (LSTM)

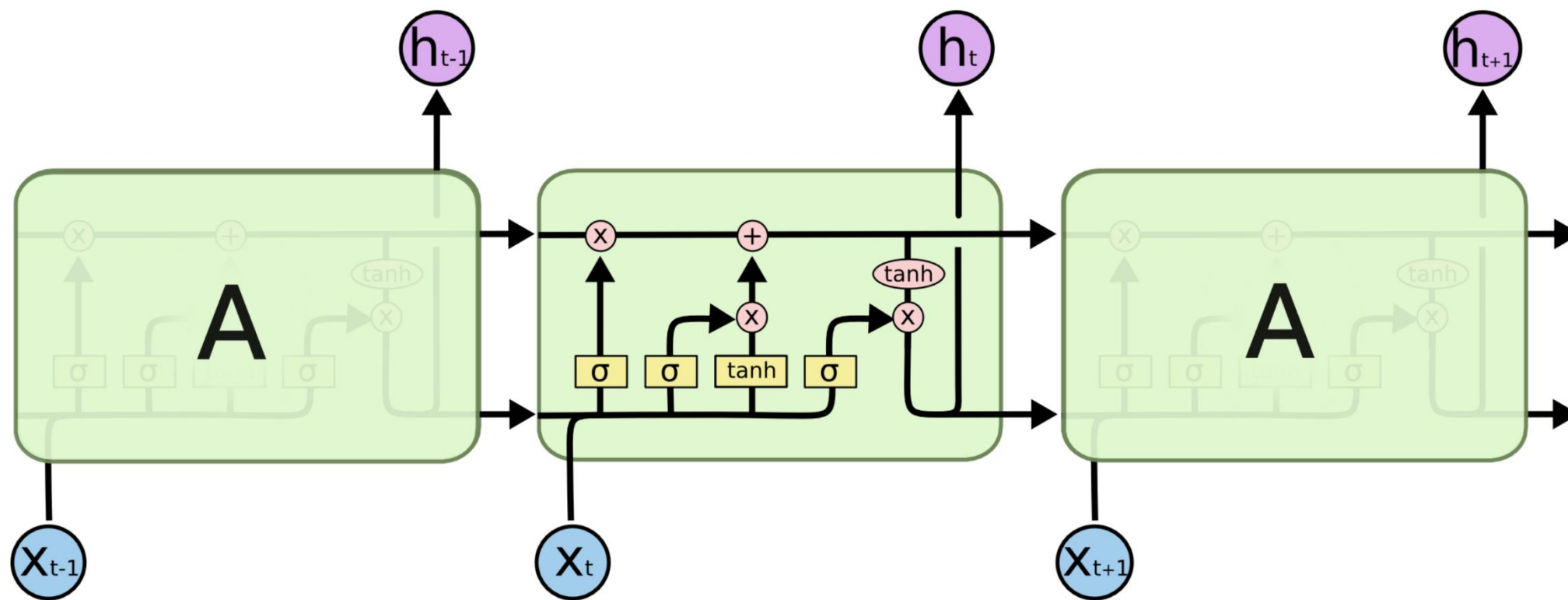


- Hochreiter & Schmidhuber (1997) solved the problem of getting an RNN to remember things for a long time (say, hundreds of time steps)
- They designed a memory cell using logistic and linear units with multiplicative interactions
 - Information gets into the cell whenever its “write” gate is on
 - Information stays in the cell so long as its “keep” gate is on
 - Information can be read from the cell by turning on its “read” gate





Repeating modules in an LSTM




Neural Network
Layer


Pointwise
Operation


Vector
Transfer


Concatenate

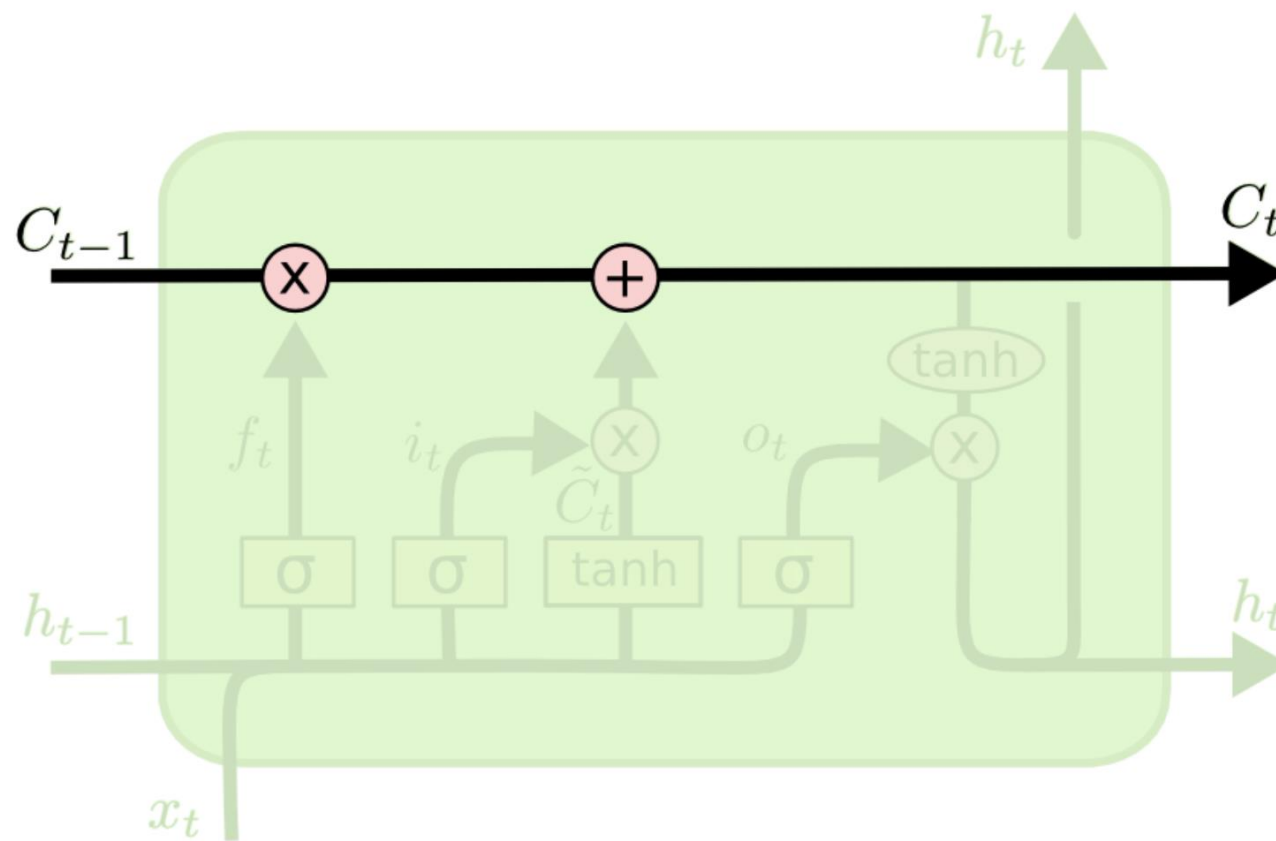

Copy



Cell state / memory



- The cell state, which holds the memory, is changed along the horizontal line running through the top of the diagram

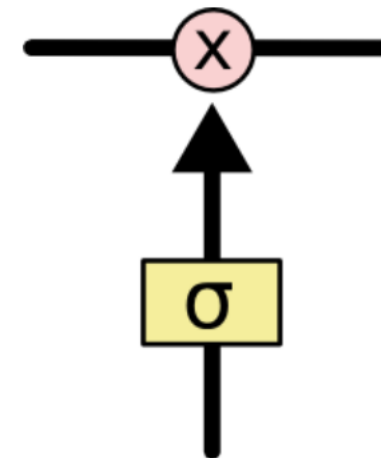




Gates

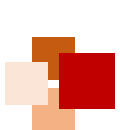


- The LSTM has the ability to remove or add information to the cell state, carefully regulated by structures called **gates**
 - Forget gate, input gate, output gate, ...
- Gates are a way to optionally let information through. They are composed of:
 - A sigmoid neural net layer, and
 - A pointwise multiplication operation
- The **sigmoid layer** outputs numbers between 0 and 1, describing how much of each component should be let through:
 - A value of **0** means “let nothing through”
 - A value of **1** means “let everything through”



σ is the sigmoid

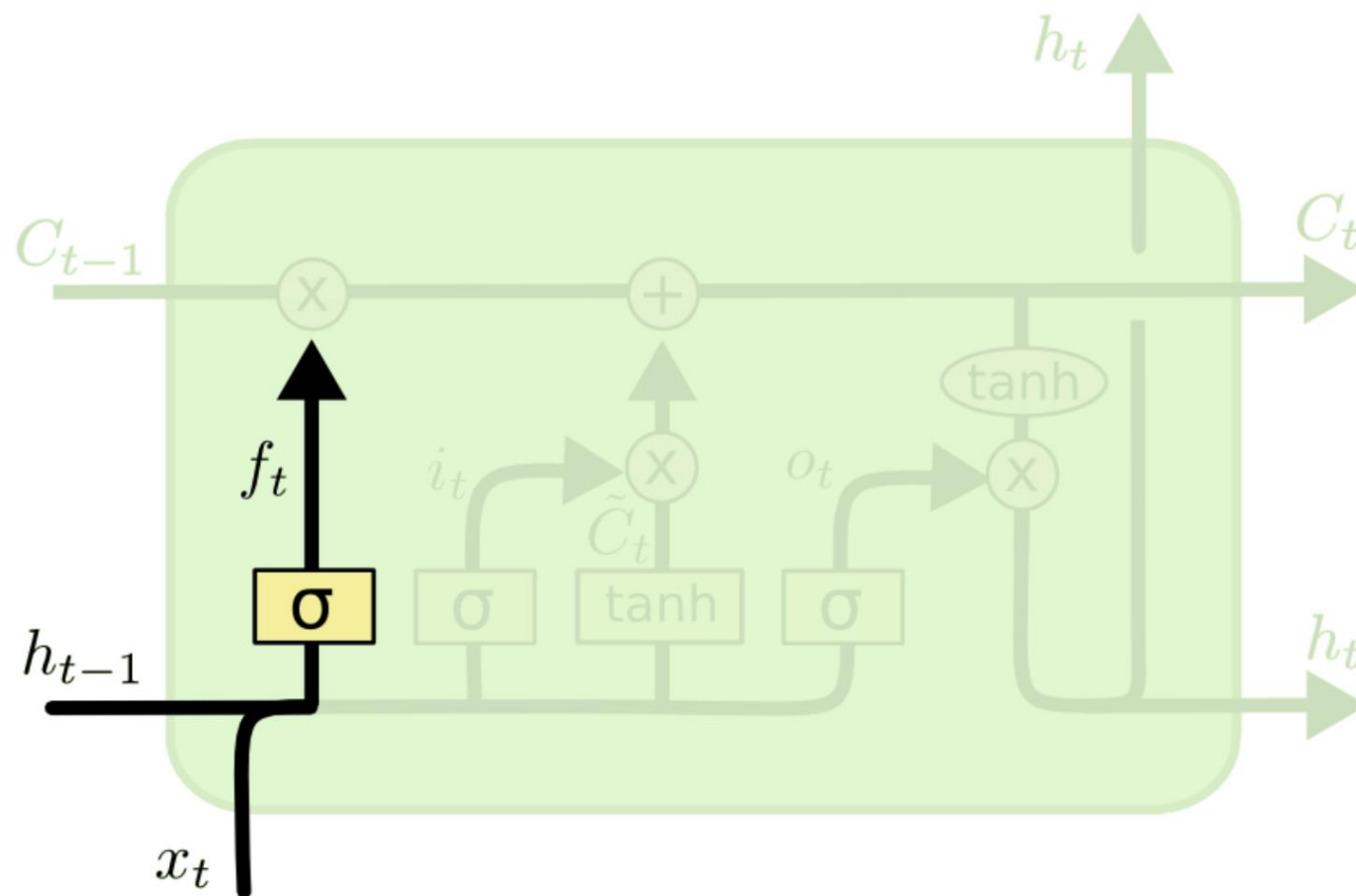




Forget gate

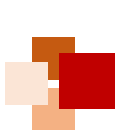


- Should we continue to remember this “bit” of information or not?



$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

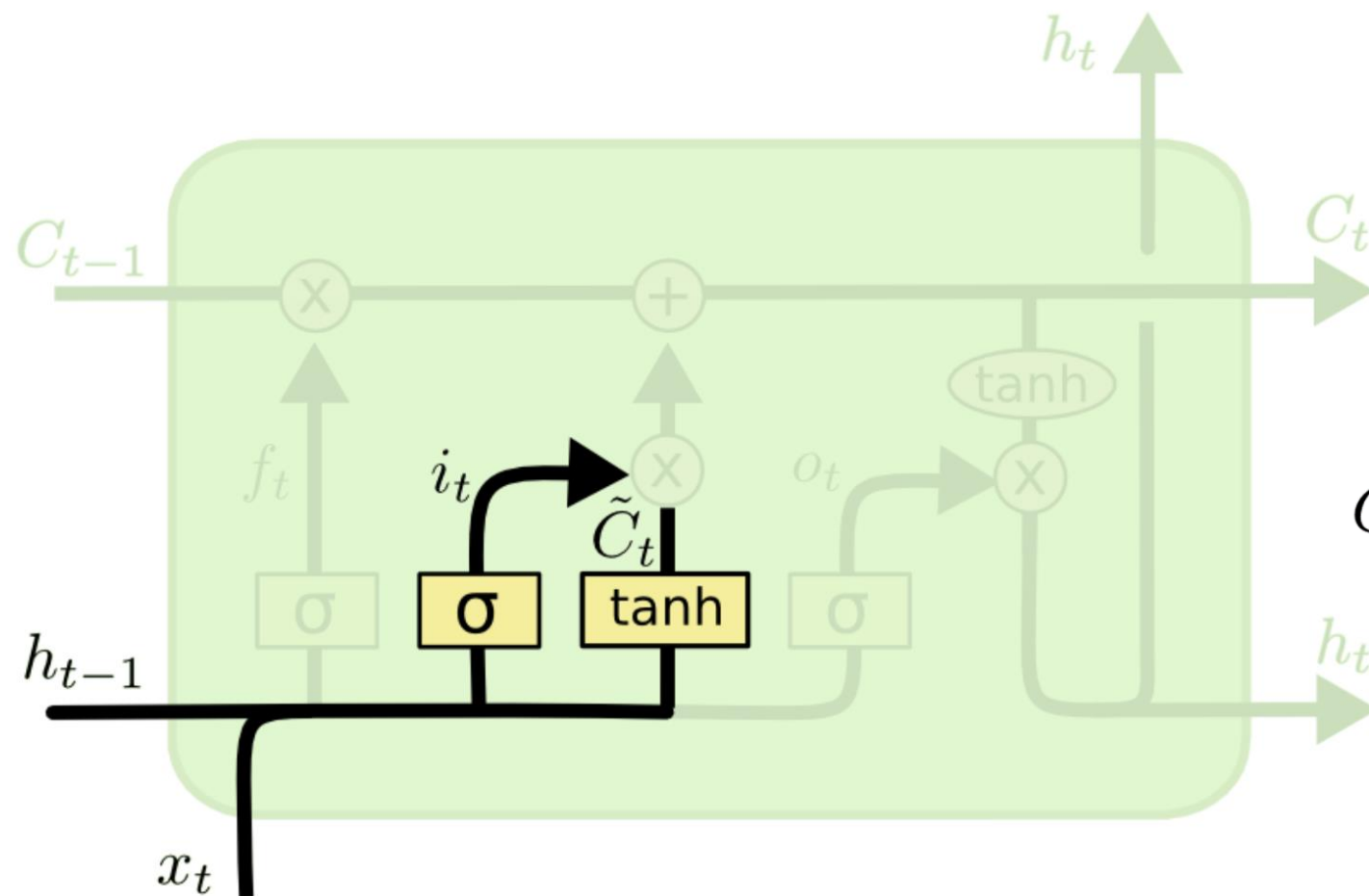




Input gate



- Should we update this “bit” of information or not? If so, with what?



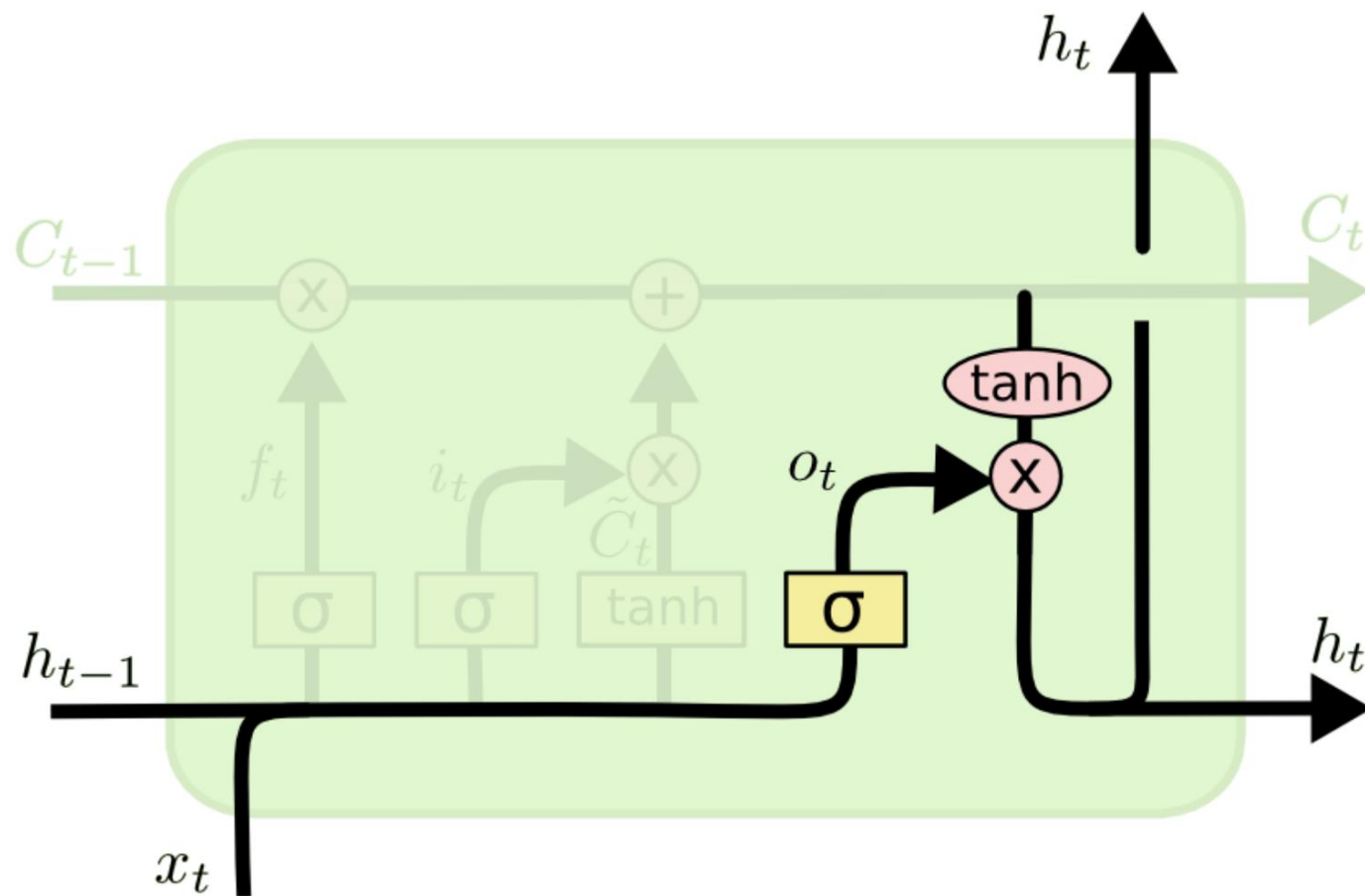
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$



Output gate

- Should we output this “bit” of information to “deeper” layers?

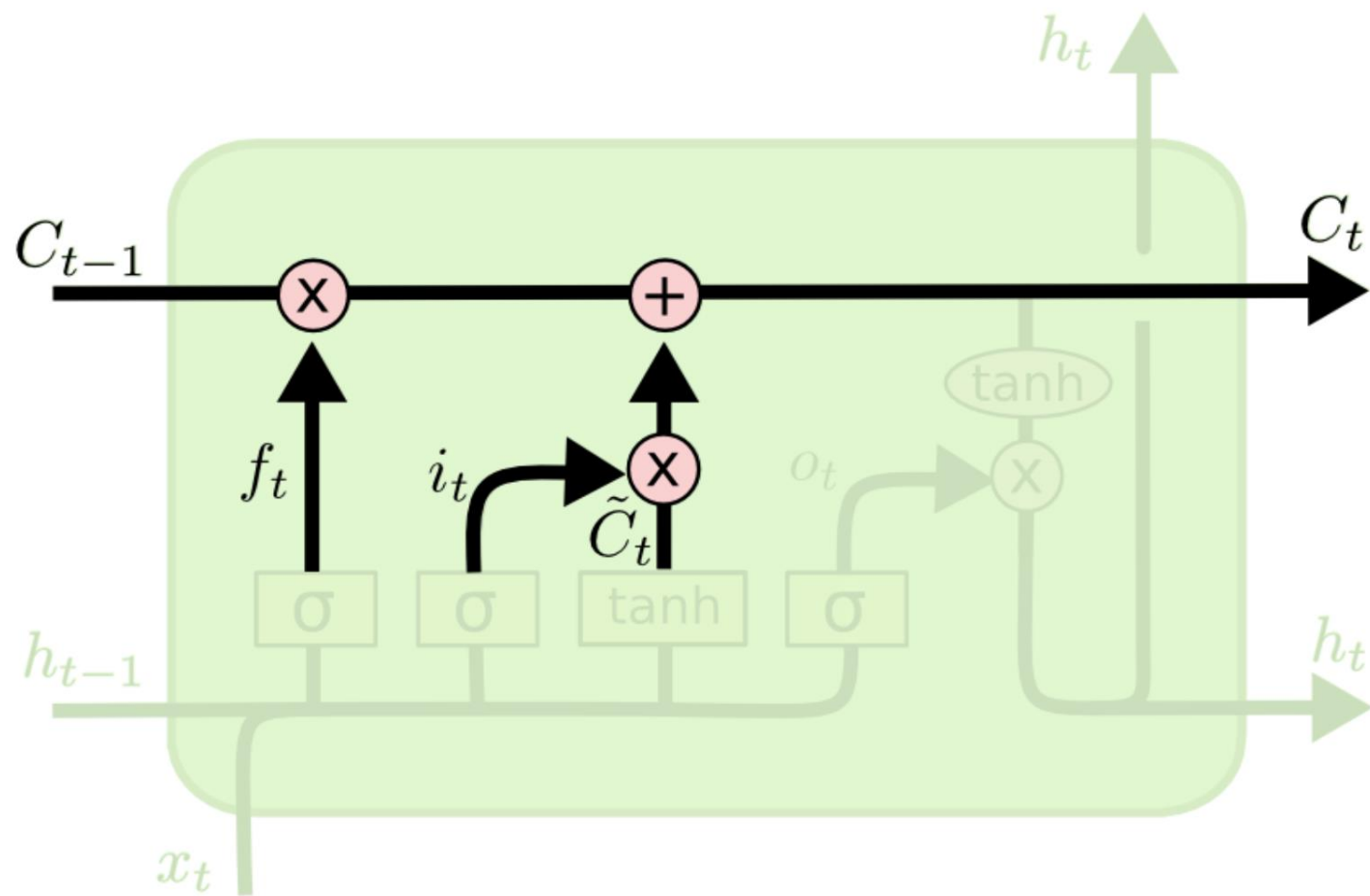


$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

Memory update

- Forget that + memorize this



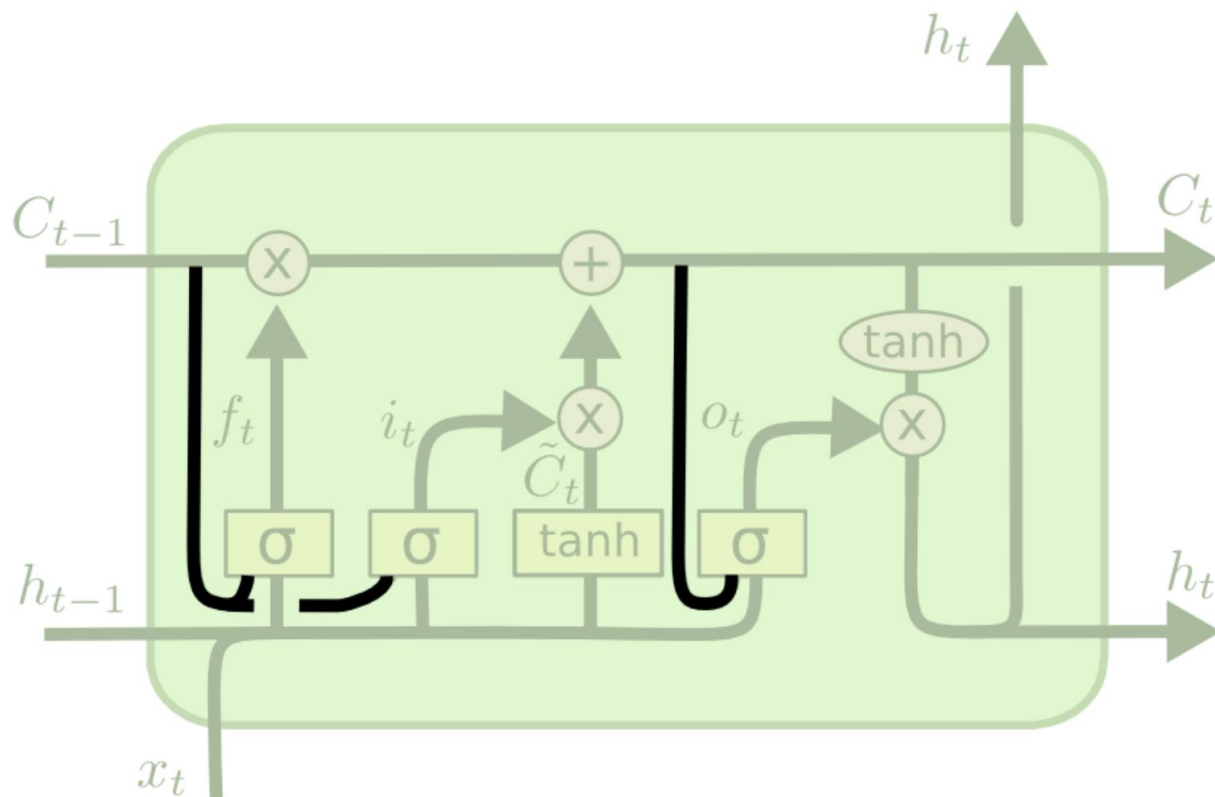
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$



Peephole connections



- To give LSTM cells more context, Gers and Schmidhuber (2000) proposed an LSTM variant with extra connections called **peephole connections**:
 - The previous long-term state C_{t-1} is added as an input to the controllers of the **forget** gate and the **input** gate
 - The current long-term state C_t is added as input to the controller of the **output** gate



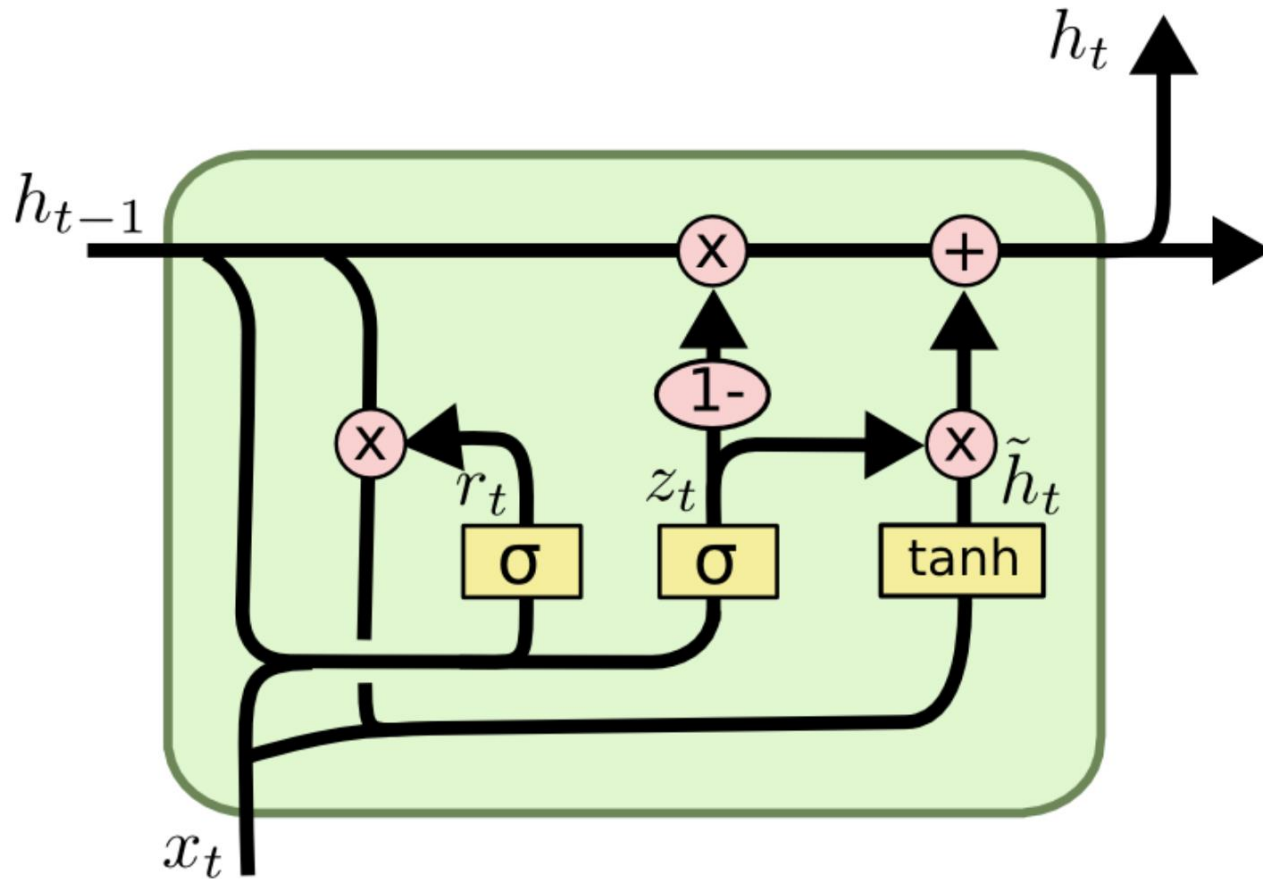
$$f_t = \sigma (W_f \cdot [C_{t-1}, h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma (W_i \cdot [C_{t-1}, h_{t-1}, x_t] + b_i)$$

$$o_t = \sigma (W_o \cdot [C_t, h_{t-1}, x_t] + b_o)$$

Gated Recurrent Units (GRUs)

- Cho et al. (2014) proposed a **simplified** version of LSTM, called **GRU**:
 - Long-term and short-term states are **merged** into a single vector h_t
 - A single gate controller controls both the forget gate and the input gate
 - There is no output gate; the full state vector is output at every time step



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$



Autoencoder (AE)





Efficient Data Representations



Question: Which sequence is easier to remember ?

- 40, 27, 25, 36, 81, 57, 10, 73, 19, 68 → The shorter sequence seems easier ?
- 50, 25, 76, 38, 19, 58, 29, 88, 44, 22, 11, 34, 17, 52, 26, 13, 40, 20 ←

What if there are some rules that we can follow ?





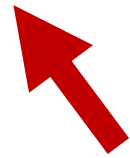
Efficient Data Representations



Question: Which sequence is easier to remember ?

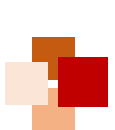
- 40, 27, 25, 36, 81, 57, 10, 73, 19, 68

- 50, 25, 76, 38, 19, 58, 29, 88, 44, 22, 11, 34, 17, 52, 26, 13, 40, 20



If you can remember the start of sequence 50,
the length of sequence and two rules, then it is
easy to reconstruct the whole sequence





Chess memory experiment

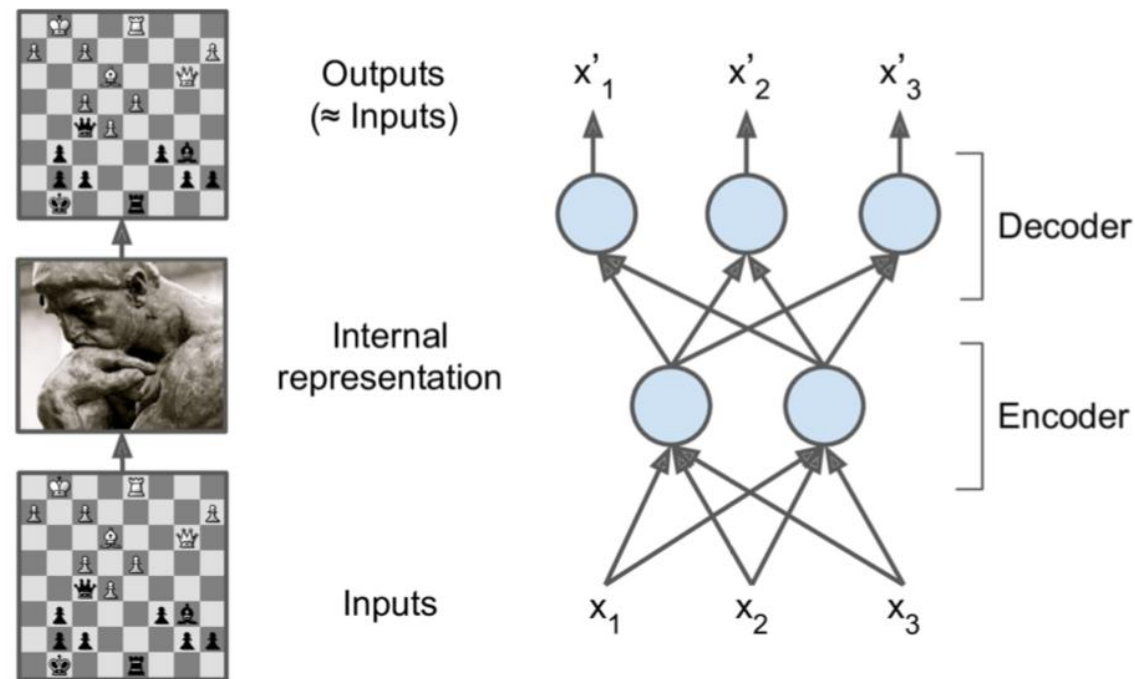


Professional chess players can memorize positions of all pieces on a chessboard in 5 seconds. How can they do that?

Step 1: Look at the chessboard

Step 2: Analyze (encode) the chessboard

Step 3: Recover (decode) the chessboard



Internal representation has a lower dimensionality than the input data

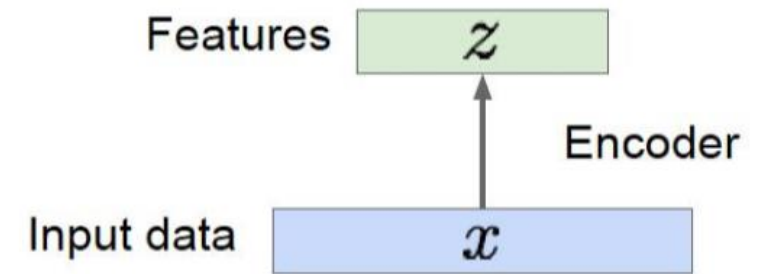




Autoencoders

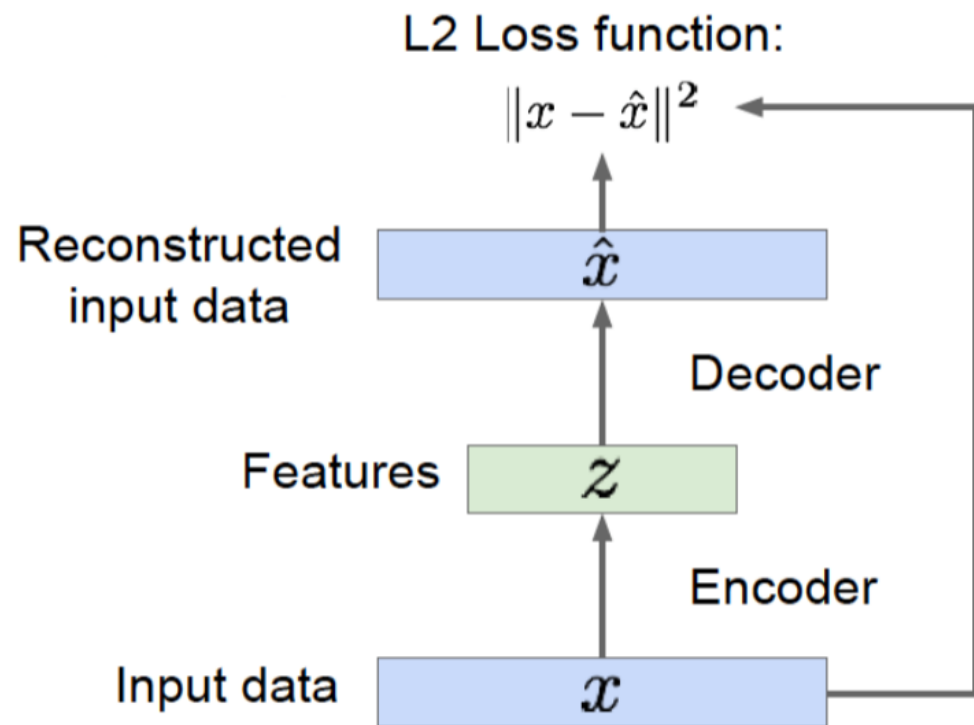


- The above two examples show that, by finding internal representations with lower dimensionality than the input data, we can find important patterns in the data
- **Autoencoder (AE)** is an **unsupervised** neural network for learning a lower-dimensional feature representation from unlabeled training data
 - It tries to reconstruct the input with lower dimensions (less neurons)



Training an Autoencoder

- How to train an autoencoder ?
 - x : input data
 - z : internal representation / feature
 - $\hat{x}$: reconstructed data
 - $\|x - \hat{x}\|^2$: **reconstruction error** (or **reconstruction loss**)
- Then, an optimizer (e.g. Adam) can be used to minimize the reconstruction loss





Principal Components Analysis (PCA)



- What is Principal Components Analysis (PCA)?
 - Reduce the data dimensionality by **linear transformation** **without losing much information**
- Formulation :
 - Suppose there are n samples in an m -dimension space $X = \{x_1, x_2, \dots, x_n\}$
 - Project the sample points into k dimension ($k < m$):
 - the projection matrix is $W = \{w_1, w_2, \dots, w_n\}$, and W is orthogonal
 - the projected points is $Z = WX$



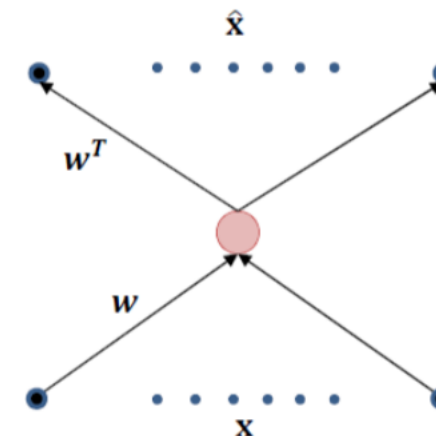


Relation between AE and PCA



- What if the encoding of AE is linear ?
 - **Encoding** process becomes a linear projection $W \in \mathbb{R}^{n \times m}$ from a high-dimensional space $x \in \mathbb{R}^n$ to a low-dimensional space $z = Wx$
 - **Decoding** process becomes $\hat{x} = W^T Wx$
 - **Reconstruction error** is $\|x - W^T Wx\|^2$, the same as PCA

- Therefore, **PCA is a special (linear) form of AE**





Example Autoencoders





Example AEs



- **Stacked (or deep) Autoencoder:** more hidden layers than standard AEs
- **Denoising Autoencoder (DAE):** reconstruct the input from a corrupted (by adding noise) version of input
- **Sparse Autoencoder:** limit the amount of total activation for a neuron throughout training
- **Variation Autoencoder (VAE):** An AE whose training is regularized
 - To avoid overfitting
 - To ensure that the latent space has good properties that enable generative process
 - Instead of encoding an input as **a single point**, VAE encodes it as **a distribution** over the latent space

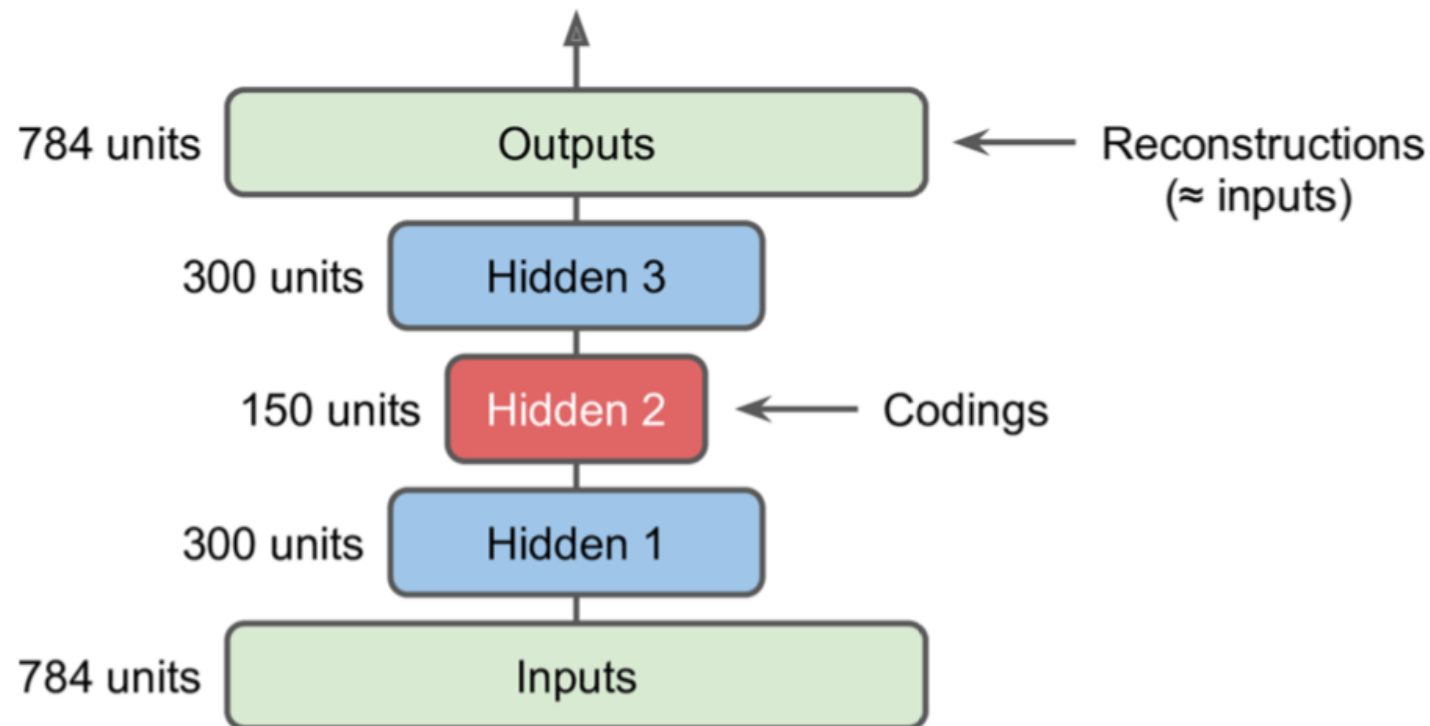


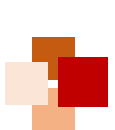


Stacked (or deep) autoencoder



- More hidden layers than standard Autoencoders

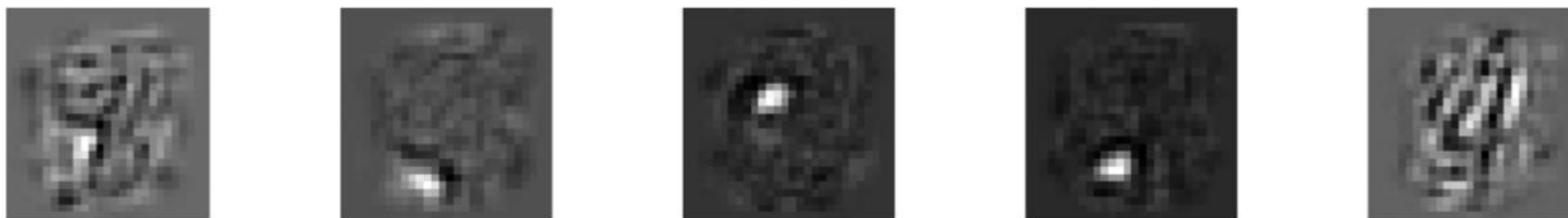




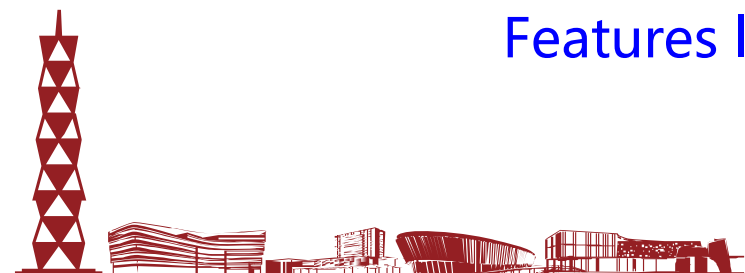
Advantages of stacked AEs



- More powerful for feature representation
 - Previous layers can extract low-level features
 - Deep (inner) layers can learn high-level features



Features learned by 5 neurons from the first hidden layer





Denoising Autoencoder (DAE)

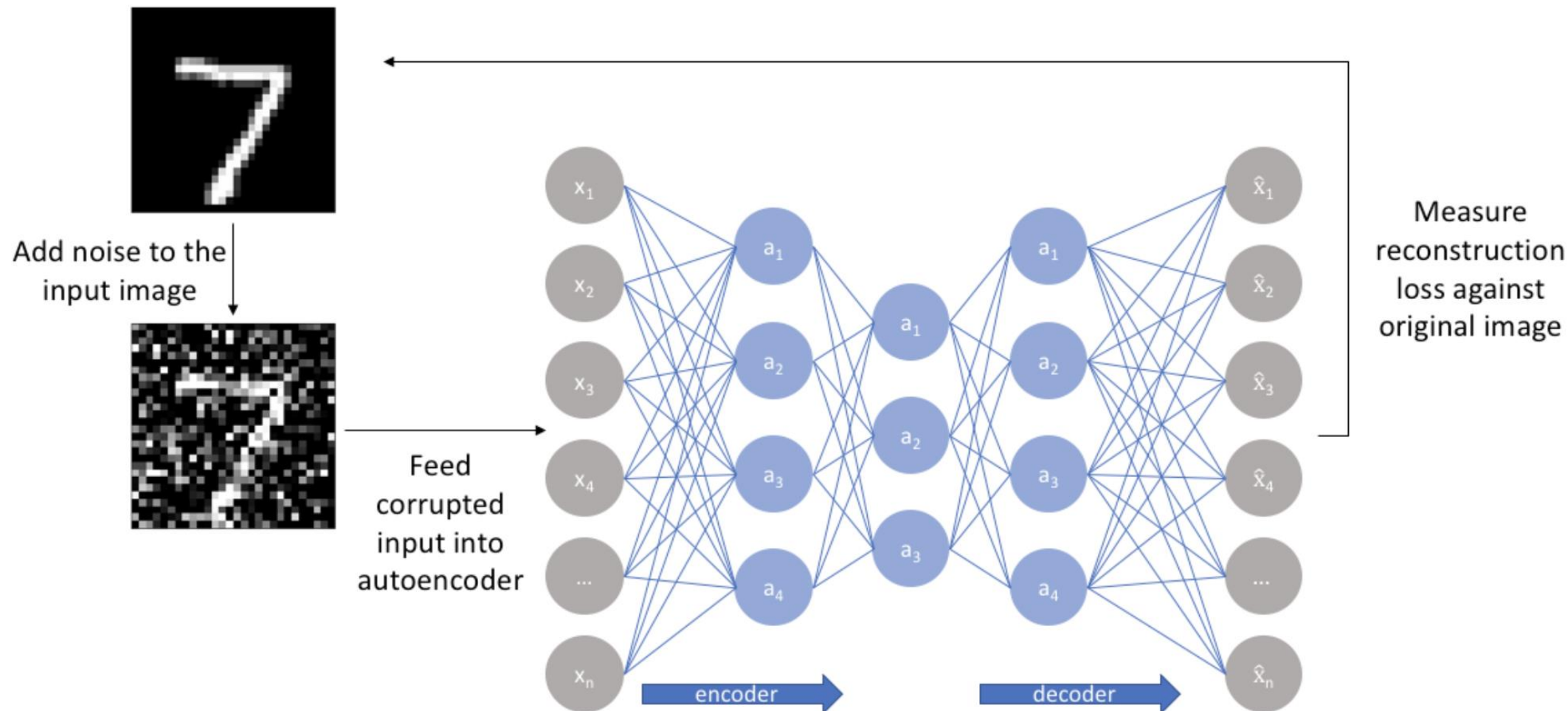


- What is DAE?
 - Randomly corrupt some of the inputs
 - Train the autoencoder to reconstruct the input from a corrupted version of it
 - The autoencoder is forced to predict the original inputs
 - This requires to capture the joint distribution between a set of variables
- **Rationale:** To increase the difficulty of reconstructing the output by adding noise

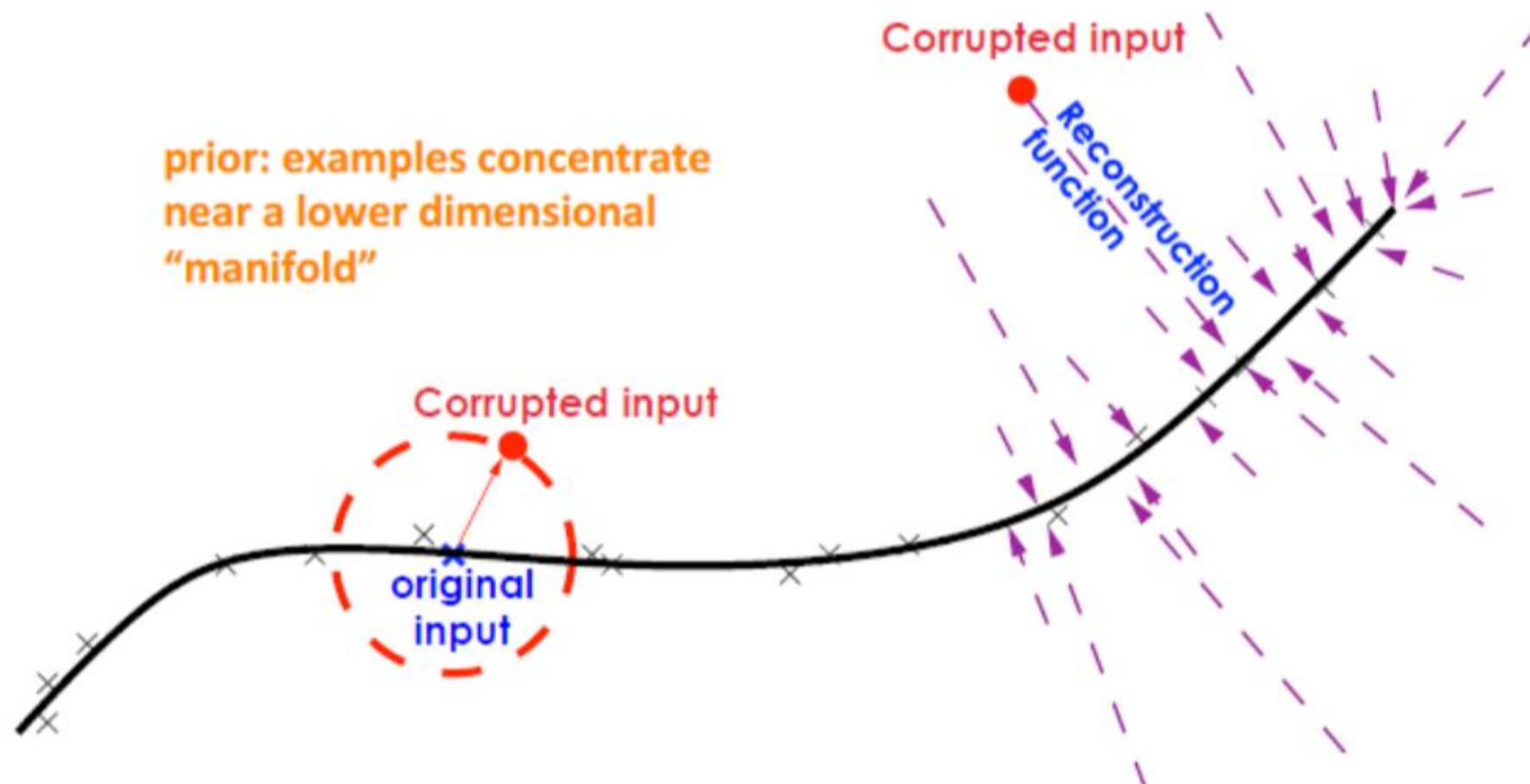




Structure of DAE



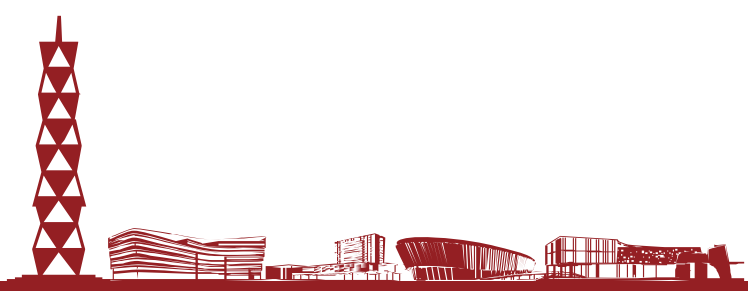
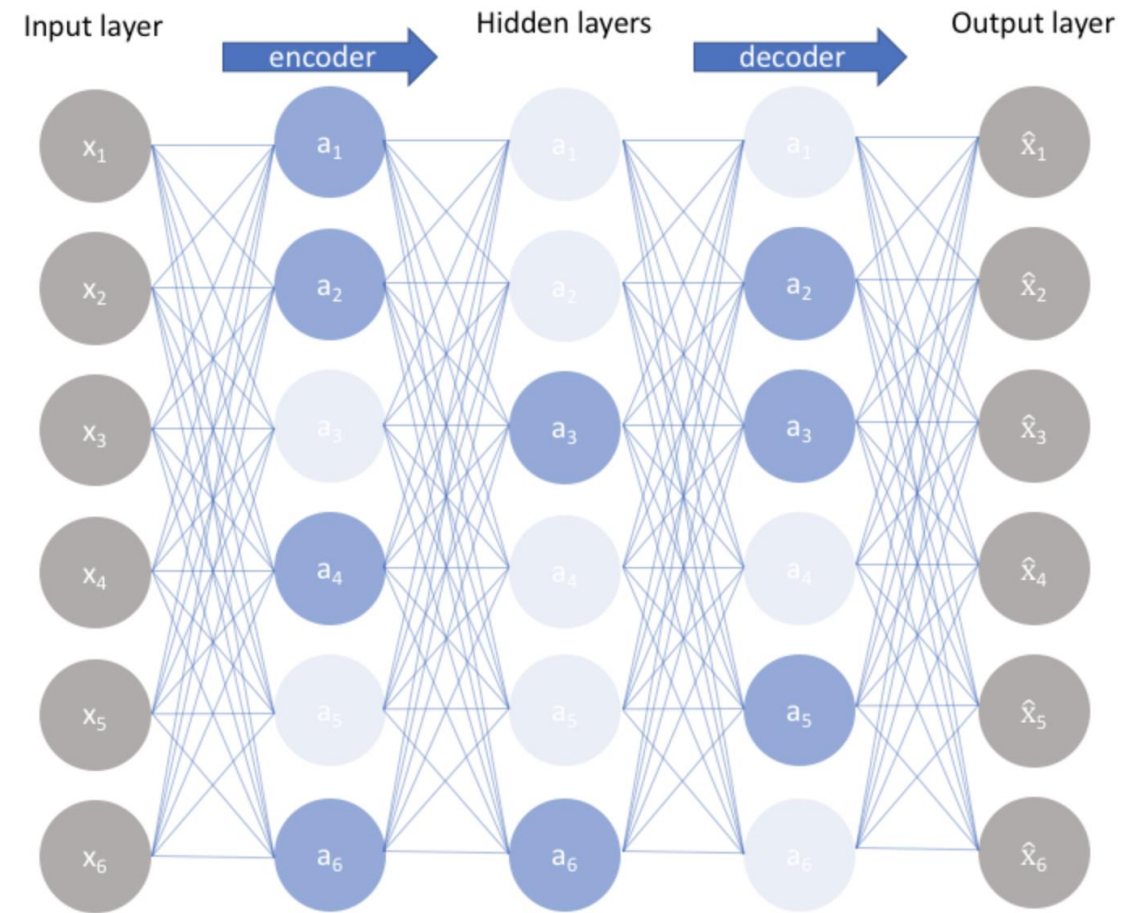
Geometric view of DAE



Sparse Autoencoder



- Usually, m the dimension of hidden layers in an AE is small
- What if the dimension of hidden layers k in the AE is large ?
 - We call it **over-completeness**
 - This can be solved by enforcing **sparsity**



- **Rationale:** To limit the amount of total activation for a neuron throughout training

$$\hat{p}_j = \frac{1}{n} \sum_{i=1}^m [a_j(x_i)]$$

- a_j : the activation of hidden unit j in the autoencoder
- $a_j(x)$: the activation of the hidden unit when the network is given a specific input x
- $\hat{p}_j$: the average activation of hidden unit j
- We enforce a **sparsity constraint** that $\hat{p}_j = p_0$
 - p_0 is small, usually set as 0.05





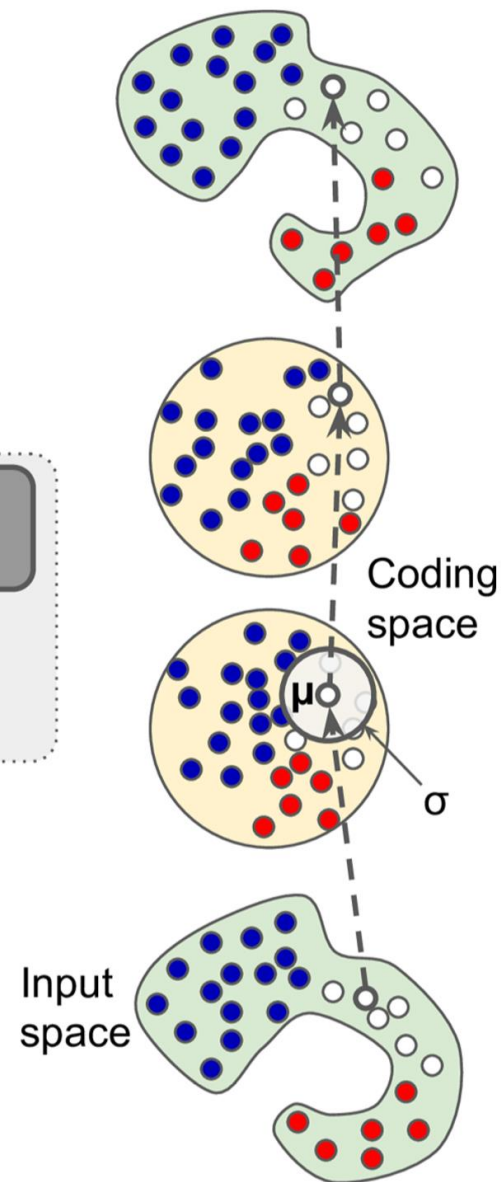
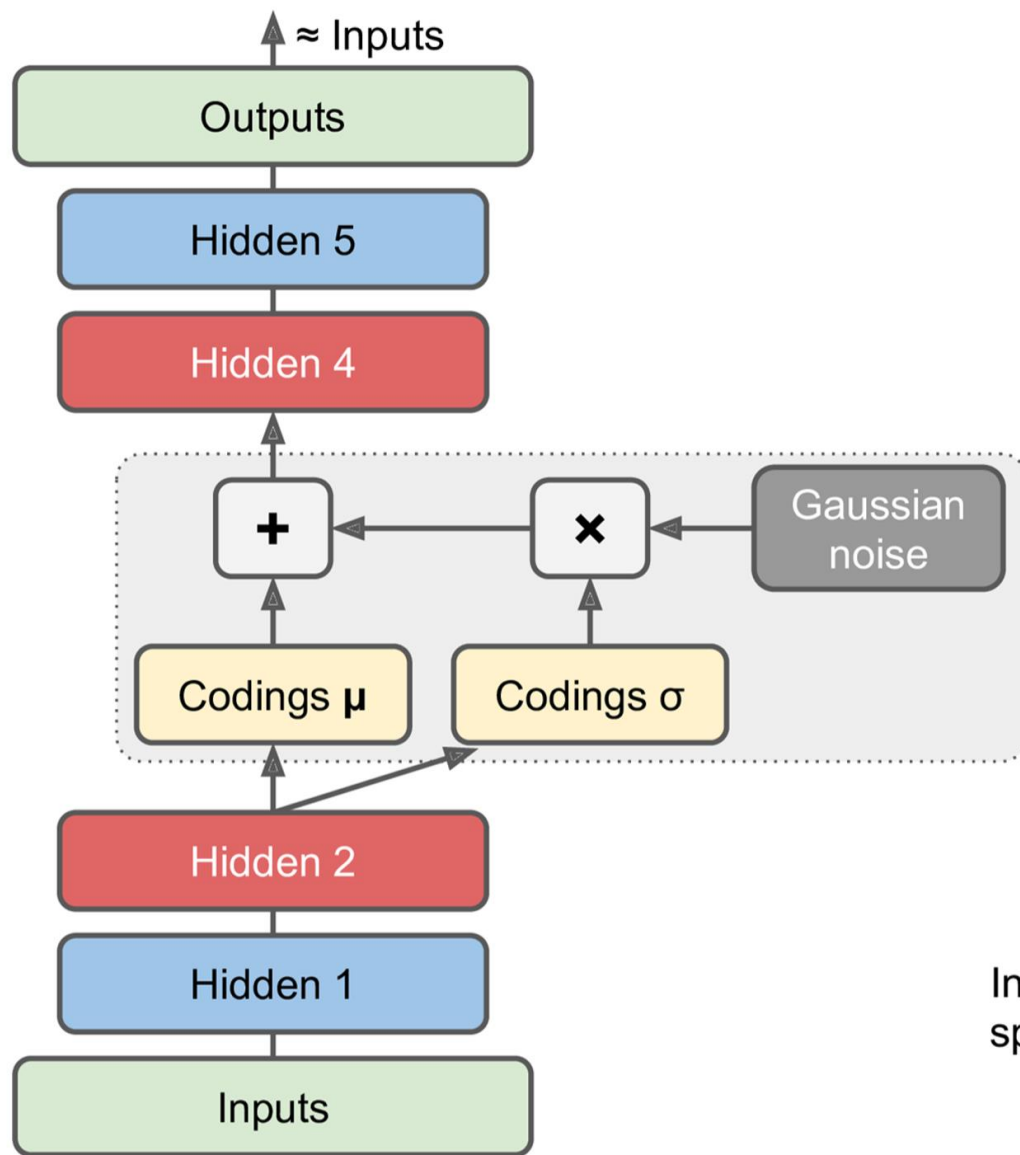
Variational Autoencoder (VAE)



- A **variational autoencoder (VAE)** can be defined as being an autoencoder whose training is regularized
 - To avoid overfitting, and
 - To ensure that the latent space has good properties that enable generative process
- Instead of encoding an input as a single point, VAE encodes it as a distribution over the latent space



Variational Autoencoder (VAE)





Recap



- In this lecture, we have learned:
 - The concept and basic architecture of RNNs
 - Training of RNNs, especially handling the unstable gradients problem
 - LSTM and GRU cells to tackle the short-term memory problem
 - The rationale of autoencoders with efficient data representations (i.e. to reconstruct inputs in a lower-dimensionality space)
 - Relation between PCA and autoencoders
 - Several variants of autoencoders
 - Stacked (or deep) AE
 - Denoising AE
 - Sparse AE
 - Variational AE





References



- Chapters 16 and 17 of Aurélien Géron' s book “Hands-On Machine Learning with SciKit-Learn, Keras & TensorFlow” (Ed. 2) , O' Reilly, 2019.
- Chapters 10 and 14 of Ian Goodfellow et al. “Deep Learning” , MIT Press, 2017.

